

Nebula MFE Architecture Documentation

Overview

Unified Runtime is a solution which brings together the low-code and hand-coded screens. It creates an ecosystem which wraps independent runtimes with minimal common features. It is capable of rendering Co-existence, Nebula Studio developed and ReactJS hand-coded screens.

Architecture

Unified Runtime consists of the following elements

1. Nebula Runtime Platform
2. Studio Runtime
3. Co-existence Runtime
4. React screens

Nebula Runtime Platform

This layer acts as the parent of all the Runtimes. Its primary functionality is to handle authentication, SaaS and other features which are independent of individual SBU implementation.

Following features are available in this layer

1. Login flow
2. Token management
3. Site settings
4. CHIA
5. General Settings
 - a. User management
 - b. Role Management
 - c. RBAC settings
 - d. Logo Upload
6. Language management
7. Role and OU context switching
8. Logout flow

All the other Runtimes are connected to Nebula Runtime Platform as Micro frontends.

Implementation in Payce

1. Repository Overview

There are **two separate repositories**, each with an `app-host` package:

Repo	Path	Role
nebula-runtime-platform	C:\Users\Adit841R N15\nebula- runtime- platform\packages \	Platform foundation — the root-app host (Module Federation Shell)
payce-mfe	C:\Projects\payce -mfe\nebula- runtime\packages\ app-host\	Product runtime — consumes root-app, exposes HRP product routes as an MFE remote

2. nebula-runtime-platform — Package Structure

```
1 packages/
2 |— api/                                ← Mock JSON server (json-server +
   auth) on :3001
3 |   |— db.json                        ← Mock database
4 |   |— routes.json                   ← Route mappings
5 |
6 |   |— app-host/                     ← ROOT-APP (Module Federation Shell)
7 |   |   |— vite.config.ts           ← Federation host config (name: 'root-
   app')
8 |   |   |— src/
9 |   |   |   |— index.tsx            ← React 18 entry point (dual Redux
   providers)
10 |   |   |   |— App.tsx              ← BrowserRouter +
   auth/tracker/preference providers
11 |   |   |   |— nebulaKernel/
12 |   |   |   |   |— redux/           ← Root Redux store
13 |   |   |   |   |   |— reduxStore.ts ← Store +
   nebulaStoreContext + exported hooks
14 |   |   |   |   |   |— reducers/
15 |   |   |   |   |   |   |— index.ts  ← combineReducers(kernel,
   product)
16 |   |   |   |   |   |   |   |— productAdapters.ts ← combineReducers(avn, erp,
   hrp, srp, logistics)
17 |   |   |   |   |   |   |   |— AppService/
18 |   |   |   |   |   |   |   |   |— AppLayout.tsx      ← Main layout (TopBar +
   UIBar + Content)
19 |   |   |   |   |   |   |   |   |   |— Content.tsx    ← Route renderer (lazy +
   Suspense + AuthorizedRoute)
20 |   |   |   |   |   |   |   |   |   |— Utils/
```

```

21 |   |   |   |   | Providers/
22 |   |   |   |   |   | authProvider.tsx
23 |   |   |   |   |   | authorizationProvider.tsx ← Exposed via
federation
24 |   |   |   |   |   | types.ts ← NebulaRouteType
definition
25 |   |   |   |   |   | components/
26 |   |   |   |   |   |   | panes/TopBar.tsx
27 |   |   |   |   |   |   | panes/UIBar.tsx
28 |   |   |   |   |   |   | useractivities/
29 |   |   |   |   |   | nebulaComponents/
30 |   |   |   |   |   |   | services/
31 |   |   |   |   |   |   | PreferenceContextProvider.tsx ← Exposed via
federation
32 |   |   |   |   |   | ramcoComponents/
33 |   |   |   |   |   |   | rui-components/ ← @nebula/authorized-rui-
components alias
34 |   |   |   |   |   |   | t-components/ ← @nebula/authorized-t-components
alias
35 |   |   |   |   |   | runtime/
36 |   |   |   |   |   |   | RuntimeRoutes.ts ← Aggregates RVW + Swift + MFE
routes
37 |   |   |   |   |   |   | rvw/ ← Rapid View Web runtime (own
Redux store)
38 |   |   |   |   |   |   | swift/ ← Swift runtime routes
39 |   |   |   |   |   |   | mfe-apps/ ← ★ Where new MFE remotes are
wired in ★
40 |   |   |   |   |   |   | MfeRoutes.tsx ← Aggregates all MFE routes
41 |   |   |   |   |   |   | sqlJobs/ ← Reference implementation
42 |   |   |   |   |   |   | index.tsx ← Bridge component (lazy loads
from remote + passes context)
43 |   |   |   |   |   |   | routes.tsx ← Route definitions for sql-jobs
44 |   |   |   |   |   |   | routes/
45 |   |   |   |   |   |   | Routes.ts ← Master route list
(UserActivities + Runtime + Saas + Product + Swift)
46 |   |   |   |   |   |   | productAdapters/ ← Product-specific store reducers
(hrp, erp, avn, srp, logistics)
47 |   |   |   |   |   |   | saas/ ← SaaS features (RBAC, user/role
management, site settings)
48 |

```

3. payce-mfe (app-host) — Package Structure

```

1 | packages/app-host/
2 |   | vite.config.ts ← MFE Remote config (name: 'hrp-
runtime-mfe')
3 |   | src/
4 |   |   | index.tsx ← Entry – dual Redux providers (Nebula
Core + local)
5 |   |   | App.tsx ← BrowserRouter + providers
6 |   |   | nebulaKernel/
7 |   |   |   | redux/
8 |   |   |   |   | reduxStore.ts ← Tries to load store from root-
app/store/hooks, falls back to local
9 |   |   |   |   | reducers/
10 |   |   |   |   | index.ts ← combineReducers(kernel,
product)
11 |   |   |   |   | productAdapters.ts ← combineReducers(avn, erp,
hrp, srp, logistics)
12 |   |   |   | productAdapters/
13 |   |   |   |   | ProductRoutes.ts ← Aggregates all product route arrays
14 |   |   |   |   | hrp/
15 |   |   |   |   | HRPRoutes.ts ← ★ Exposed via federation as
'./HRPRoutes' ★

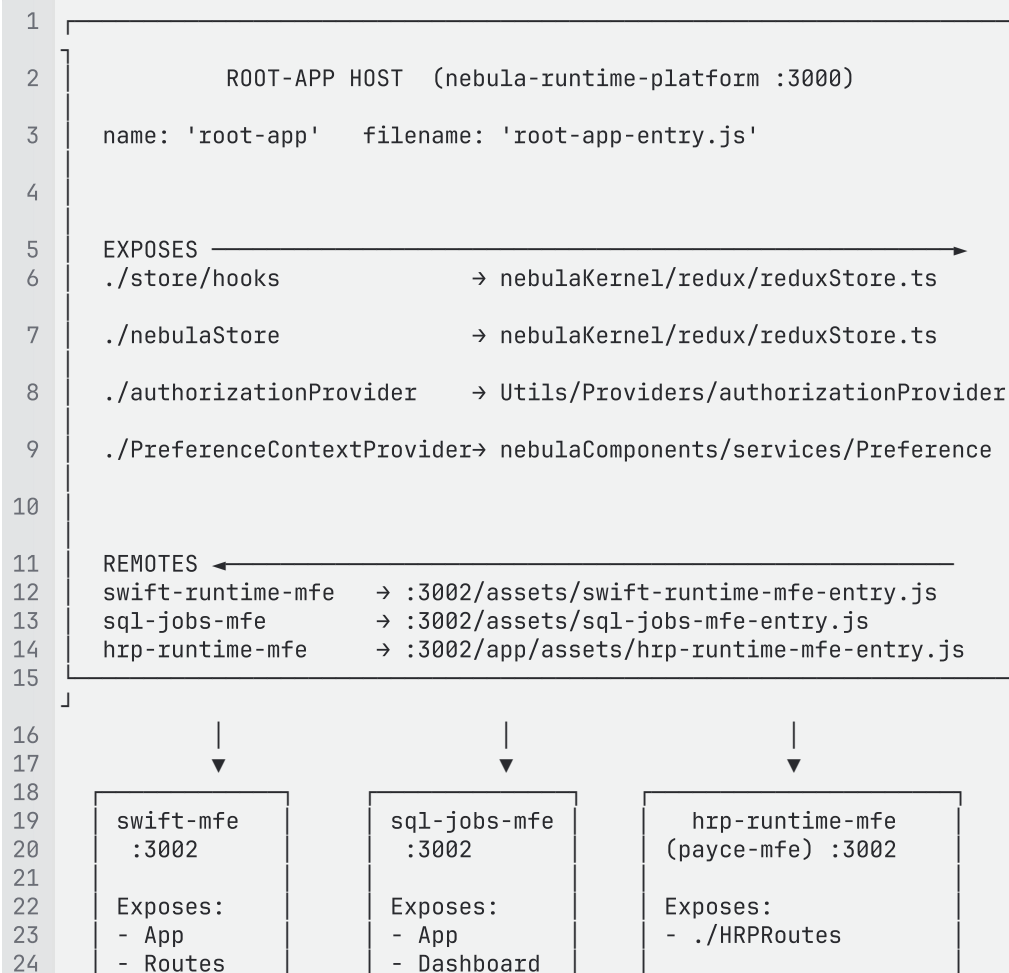
```

```

16 | constants.ts    ← SCREEN_INFO, TASK_NAMES,
   | HRP_URL_DICT
17 | store/
18 |   reducers/
19 |     index.ts    ← combineReducers(IP0,
payrollWorkspace, QIK,
20 | LandingTemplate, ActivityProgress,
21 | TaxDeclaration, SelectedTab)
22 |   activityProgress.ts
23 |   actions/
24 |     progressActivity.ts
25 |     constants/index.ts
26 |   Modules/
27 |     Implementation/ ← Implementer landing, activity
badge, stepper
28 |   IP0/            ← Rule engine, configurator,
entity details
29 |     ESS/          ← Self-service, tax declaration
30 |     PayrollHub/
31 |   exp/
32 |   avn/
33 |   srp/
34 |   logistics/
35 |   runtime/        ← RVW runtime (copy from platform)
36 |

```

4. Module Federation Architecture



Remote URL Resolution Logic

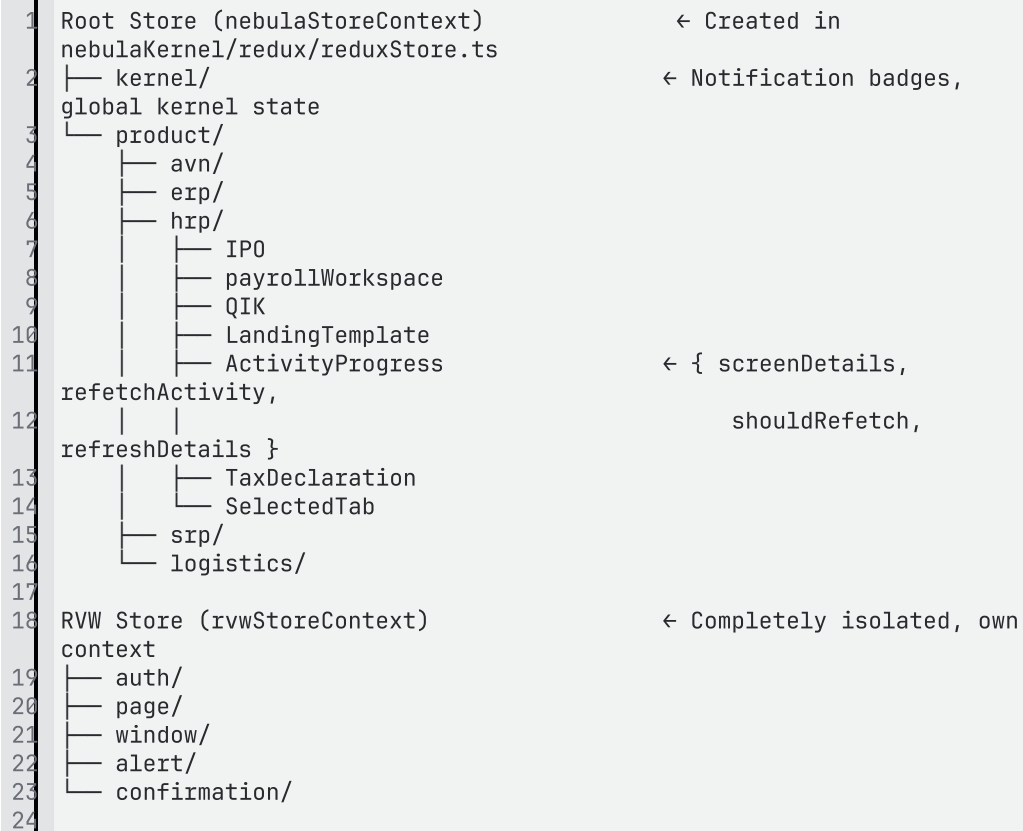
Remote	Dev (localhost)	Production
swift-runtime-mfe	http://localhost:3002/assets/swift-runtime-mfe-entry.js	{origin}/swift-app/assets/swift-runtime-mfe-entry.js
sql-jobs-mfe	http://localhost:3002/assets/sql-jobs-mfe-entry.js	{origin}/mfe-apps/sql-jobs/assets/sql-jobs-mfe-entry.js
hrp-runtime-mfe	http://localhost:3002/app/assets/hrp-runtime-mfe-entry.js	{origin}/mfe-apps/hrp-app/assets/hrp-runtime-mfe-entry.js

hrp-runtime-mfe additionally injects `window.__hrpRuntimeMfeAssetPath__` before resolving, so the remote can correctly load its own assets.

Shared Libraries (Singletons across all MFEs)

```
1  react                react-dom
2  react-router-dom     react-redux
3  zustand              react-query
4  react-i18next         i18next
5  react-toastify        react-multi-date-picker
6  reactflow            @ramco-platform/core
7  @ramco-platform/studio-components
8  @ramco-platform/t-components
9  @bryntum/calendar     @bryntum/calendar-react
10 @bryntum/gantt         @bryntum/gantt-react
11 @bryntum/schedulerpro  @bryntum/schedulerpro-react
12 html-to-draftjs       botframework-webchat
13
```

5. Redux Store Hierarchy



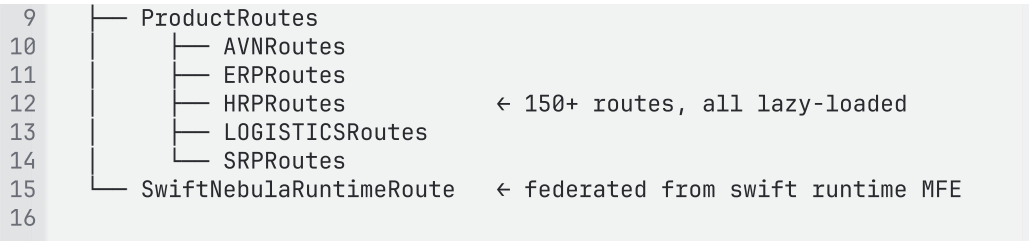
Selector Path Reference

```
1 // Accessing hrp ActivityProgress from a component
2 useNebulaCoreSelector((state: any) =>
  state?.product?.hrp?.ActivityProgress ?? {})
3
4 // Accessing kernel state
5 useNebulaCoreSelector((state: any) => state?.kernel)
6
7 // RVW store (different context – use useRvwSelector)
8 useRvwSelector((state: any) => state?.auth)
9
```

Important: Always provide a `?? {}` fallback when destructuring from a selector — the slice may not be initialized yet on first render.

6. Route Flow

```
1 Routes.ts
2 └─ UserActivitiesRoutes
3 └─ RuntimeRoutes
4   └─ RVW_ROUTES (only if AppSettings.enableCoExistence
= true)
5   └─ SwiftRTRoute (only if AppSettings.enableCoExistence
= true)
6   └─ MfeRoute ← always included
7     └─ SqlJobsRoute ← /sql-jobs, /sql-jobs/dashboard
8 └─ SaasRoutes
```



NebulaRouteType Shape

```

1  type NebulaRouteType = {
2    path: string;                // URL path (e.g. '/sql-jobs')
3    url: () => Promise<any>;      // Dynamic import of the component
4    loadable: boolean;           // true = lazy load via React.lazy
5  + Suspense
6    uiName?: string;             // Screen identifier
7    activityName?: string;       // Activity name for authorization
8    componentName?: string;     // Component name for
9    authorization
10   ilboCode?: string;           // ILBO code for RBAC
11   enableRBAC?: boolean;       // false = skip role-based access
12   check
13 };
14
```

7. Context Bridge Pattern (How MFEs Receive Context)

Since MFEs live in separate bundles loaded at runtime, the host bridges context via `window CustomEvents` from the bridge component (`index.tsx` in `mfe-apps/{mfe}/`).

Events Dispatched by Host → Remote

Event Name	Payload	Trigger
USER_CONTEXT	userContext , pageContext , menuContext , authContext , settings , appSettings , navData , dialogInfo , drawerInfo , precisionContext , preferenceContext , componentName , activityName , AuthorizationCont	On mount & when userContext changes

	<code>ext ,</code> <code>useNavigationServ</code> <code>ice</code>	
<code>AUTHORIZATION_CON</code> <code>TEXT</code>	<code>controls ,</code> <code>disallowedTasks</code>	When <code>controls</code> or <code>disallowedTasks</code> changes
<code>PREFERENCE_CONTEX</code> <code>T</code>	<code>preferences</code>	When preferences change
<code>CLIENT_SERVICES_C</code> <code>ONTEXT</code>	<code>invokeApi ,</code> <code>downloadApi ,</code> <code>useGQLQuery ,</code> <code>useGQLMutation ,</code> <code>HttpMethodMap</code>	When API services are available

Event Dispatched by Remote → Host

Event Name	Payload	Purpose
<code>LOG_OUT</code>	<code>{ logout: true }</code>	Triggers full logout flow in the host

8. Steps to Set Up a New MFE Remote (e.g., `my-new-mfe`)

Follow every step in order across both the **host** (`nebula-runtime-platform`) and the **new remote project**.

Step 1 — Register the Remote in the Host's `vite.config.ts`

File: `nebula-runtime-platform/packages/app-host/vite.config.ts`

Add a new entry inside the `remotes` block of the `federation()` plugin:

```

1 remotes: {
2   // ... existing remotes
3
4   'my-new-mfe': {
5     external: `new Promise(resolve => {
6       const windowOrigin = window.location.origin;
7       if (["http://localhost:3000",
8         "http://localhost:3001"].includes(windowOrigin)) {
9         resolve("http://localhost:3002/assets/my-new-mfe-entry.js")
10      }
11    })`
12   }
13 }

```



```

9      } else {
10         resolve(windowOrigin + "/mfe-apps/my-new-app/assets/my-new-
mfe-entry.js")
11      }
12    })`,
13    externalType: 'promise',
14  },
15 },
16

```

The key `'my-new-mfe'` **must exactly match** the `name` field in the remote's own `vite.config.ts`.

Step 2 — Create the MFE Folder in `mfe-apps/`

Create a new folder under `src/runtime/mfe-apps/`:

```

1  src/runtime/mfe-apps/
2  └─ myNewMfe/
3     └─ index.tsx    ← Bridge component
4     └─ routes.tsx   ← Route definitions
5

```

Step 3 — Create `routes.tsx`

```

1  // src/runtime/mfe-apps/myNewMfe/routes.tsx
2
3  const MyNewMfeRoutes: any[] = [
4    {
5      path: '/my-new-mfe',
6      url: () => import('./index'),
7      loadable: true,
8      uiName: 'MyNewMfe',
9      enableRBAC: false,
10   },
11   {
12     path: '/my-new-mfe/some-page',
13     url: () => import('./index'),
14     loadable: true,
15     uiName: 'SomePage',
16     enableRBAC: false,
17   },
18 ];
19
20 export default MyNewMfeRoutes;
21

```

`uiName` should match what the remote uses to identify the screen to render. Set `enableRBAC: false` during initial development to bypass role-based access checks.

Step 4 — Create `index.tsx` (Bridge Component)

This is the most critical file. It lazy-loads the federated component and bridges all host context to the remote via `window CustomEvents`.

```
1 // src/runtime/mfe-apps/myNewMfe/index.tsx
2
3 import { lazy, useContext, useEffect } from 'react';
4 import {
5   AppSettings,
6   store,
7   invokeApi,
8   HttpMethodMap,
9   downloadApi,
10  useGQLMutation,
11  useGQLQuery,
12 } from '@ramco-platform/core';
13 import { ToastContainer } from 'react-toastify';
14 import { AuthorizationContext } from
15   'nebulaKernel/Utils/Providers/authorizationProvider';
16 import { cleanSessionStorageContextInfo } from
17   '../nebulaKernel/components/panes/Utils/rvwUtils';
18 import { AuthContext } from
19   '../nebulaKernel/Utils/Providers/authProvider';
20 import rvwStore from '@nebula/rvw-store/reduxStore';
21 import { useParams } from 'react-router-dom';
22 import { PreferenceContext } from 'nebulaComponents/services';
23 import { useNavigationService } from
24   '../nebulaKernel/services/useNavigationService';
25
26 // * Lazy-load from the federated remote
27 // 'my-new-mfe' must match the key in vite.config.ts remotes
28 // 'App' must match a key in the remote's exposes config
29 const MyNewMfeApp = lazy(() => import('my-new-mfe/App') as any);
30
31 export const MyNewMfe = (props: any) => {
32   const {
33     userContext, menuContext, pageContext, settings,
34     authContext, preferenceContext, navData,
35     dialogInfo, drawerInfo, precisionContext,
36   } = store.getState() as any;
37
38   const context = useContext(AuthContext);
39   const { controls, disallowedTasks } = useContext<any>
40     (AuthorizationContext);
41   const preferences = useContext<any>(PreferenceContext);
42   const params: any = useParams();
43   const { componentName, activityName } = params;
44
45   // Bridge: User + Page context
46   useEffect(() => {
47     if (userContext) {
48       window.dispatchEvent(new CustomEvent('USER_CONTEXT', {
49         detail: {
50           userContext, pageContext, menuContext, authContext,
51           precisionContext, preferenceContext, settings,
52           navData, dialogInfo, drawerInfo,
53           componentName, activityName,
54           AuthorizationContext,
55           appSettings: AppSettings,
56           useNavigationService,
57         },
58       }));
59     }
60   });
61 }
```

```

56     const handleLogout = (input: any) => {
57         if (input.detail?.logout) {
58             localStorage.removeItem('signIn');
59             context.logout();
60             cleanSessionStorageContextInfo();
61             localStorage.removeItem('persist:root');
62             if (
63                 window.location?.pathname?.includes('rvw') ||
64                 rvwStore?.getState()?.auth?.loginInfo
65             ) {
66                 (globalThis as any).rvwEvent.doLogout();
67             }
68             window.removeEventListener('LOG_OUT', handleLogout);
69         }
70     };
71
72     window.addEventListener('LOG_OUT', handleLogout);
73 }, []);
74
75 // Bridge: Authorization context
76 useEffect(() => {
77     if (controls || disallowedTasks) {
78         window.dispatchEvent(new CustomEvent('AUTHORIZATION_CONTEXT', {
79             detail: { controls, disallowedTasks },
80             }));
81     }
82 }, [controls, disallowedTasks]);
83
84 // Bridge: Preference context
85 useEffect(() => {
86     window.dispatchEvent(new CustomEvent('PREFERENCE_CONTEXT', {
87         detail: { preferences },
88         }));
89 }, [preferences]);
90
91 // Bridge: API services
92 useEffect(() => {
93     if (downloadApi && invokeApi && HttpMethodMap && useGQLQuery &&
94     useGQLMutation) {
95         window.dispatchEvent(new CustomEvent('CLIENT_SERVICES_CONTEXT',
96         {
97             detail: { invokeApi, downloadApi, useGQLQuery,
98             useGQLMutation, HttpMethodMap },
99             }));
100     }
101 }, [downloadApi, invokeApi, HttpMethodMap, useGQLQuery,
102 useGQLMutation]);
103
104 return (
105     <div className="swift-runtime-container">
106         <MyNewMfeApp />
107         <ToastContainer />
108     </div>
109 );
110 };
111 };

```

Step 5 — Register Routes in MfeRoutes.tsx

File: src/runtime/mfe-apps/MfeRoutes.tsx

```

1 import SqlJobsRoute from './sqlJobs/routes';
2 import MyNewMfeRoutes from './myNewMfe/routes'; // ← add import
3

```

```

4 const MfeRoute: any[] = [
5   ...SqlJobsRoute,
6   ...MyNewMfeRoutes, // ← spread new routes
7 ];
8
9 export default MfeRoute;
10

```

Step 6 — Configure the Remote Project's `vite.config.ts`

In the **remote MFE project**, configure Module Federation to expose its components and declare shared libraries:

```

1 // vite.config.ts of the remote project
2
3 federation({
4   name: 'my-new-mfe',           // ★ Must match the key in host's
  remotes
5   filename: 'my-new-mfe-entry.js', // ★ Must match the filename in the
  URL
6
7   // What this remote exposes to the host
8   exposes: {
9     './App': './src/App.tsx',
10    './Dashboard': './src/pages/Dashboard.tsx', // add more pages as
  needed
11  },
12
13  // Must align with host's shared array to share singleton instances
14  shared: [
15    'react',
16    'react-dom',
17    'react-router-dom',
18    'react-redux',
19    'react-query',
20    'react-i18next',
21    'i18next',
22    'zustand',
23    '@ramco-platform/core',
24    '@ramco-platform/studio-components',
25    '@ramco-platform/t-components',
26    'react-toastify',
27  ],
28
29  // Consume host's exposed modules if needed
30  remotes: {
31    'root-app': {
32      external: `new Promise(resolve => {
33        resolve(window.location.origin + "/app/assets/root-app-
  entry.js")
34      })`,
35      externalType: 'promise',
36    },
37  },
38  }),
39

```

Step 7 — Consume Host Context in the Remote

In the remote app, listen to the CustomEvents dispatched by the host bridge:

```

1 // e.g. src/context/HostContextProvider.tsx (in the remote project)
2
3 useEffect(() => {
4   const handleUserContext = (event: CustomEvent) => {
5     const {
6       userContext, appSettings, menuContext,
7       authContext, preferenceContext, useNavigationService,
8     } = event.detail;
9     // store in local state, context, or zustand
10  };
11
12  const handleAuthContext = (event: CustomEvent) => {
13    const { controls, disallowedTasks } = event.detail;
14  };
15
16  const handlePreferenceContext = (event: CustomEvent) => {
17    const { preferences } = event.detail;
18  };
19
20  const handleClientServices = (event: CustomEvent) => {
21    const { invokeApi, downloadApi, useGraphQLQuery, HttpMethodMap } =
22    event.detail;
23  };
24
25  window.addEventListener('USER_CONTEXT', handleUserContext as
26  EventListener);
27  window.addEventListener('AUTHORIZATION_CONTEXT', handleAuthContext
28  as EventListener);
29  window.addEventListener('PREFERENCE_CONTEXT',
30  handlePreferenceContext as EventListener);
31  window.addEventListener('CLIENT_SERVICES_CONTEXT',
32  handleClientServices as EventListener);
33
34  return () => {
35    window.removeEventListener('USER_CONTEXT', handleUserContext as
36    EventListener);
37    window.removeEventListener('AUTHORIZATION_CONTEXT',
38    handleAuthContext as EventListener);
39    window.removeEventListener('PREFERENCE_CONTEXT',
40    handlePreferenceContext as EventListener);
41    window.removeEventListener('CLIENT_SERVICES_CONTEXT',
42    handleClientServices as EventListener);
43  };
44  }, []);

```

To trigger a logout from within the remote:

```

1 window.dispatchEvent(new CustomEvent('LOG_OUT', { detail: { logout:
2   true } }));

```

Step 8 — (Optional) Add a Redux Reducer for MFE State in the Host

If the new MFE needs to share state through the host's Nebula Core Redux store (like `ActivityProgress` does for HRP), add a reducer:

File: `src/nebulaKernel/redux/reducers/productAdapters.ts`

```

1 import myNewMfeReducers from
2   '../..../productAdapters/myNewMfe/store/reducers';

```

```

2
3 export default combineReducers({
4   avn: avnReducers,
5   erp: erpReducers,
6   hrp: hrpReducers,
7   srp: srpReducers,
8   logistics: logisticsReducers,
9   myNewMfe: myNewMfeReducers, // ← add
10 });
11

```

Access it in any component using:

```

1 // In the host or MFE (via root-app/store/hooks)
2 useNebulaCoreSelector((state: any) =>
3   state?.product?.myNewMfe?.SomeSlice ?? {});

```

Checklist Summary

#	File	Action
1	Host vite.config.ts	Add entry to remotes with promise-based URL resolution
2	Host src/runtime/mfe-apps/myNewMfe/	Create the folder
3	Host myNewMfe/routes.tsx	Define route paths, lazy import, uiName, enableRBAC
4	Host myNewMfe/index.tsx	Bridge component — lazy-loads remote, dispatches 4 CustomEvents
5	Host MfeRoutes.tsx	Import and spread new routes
6	Remote vite.config.ts	Set name, filename, exposes, shared; consume root-app

7	Remote App	Listen to USER_CONTEXT , AUTHORIZATION_CON TEXT , PREFERENCE_CONTEX T , CLIENT_SERVICES_C ONTEXT
8	Host productAdapters.t s (optional)	Add reducer if MFE needs shared Redux state via Nebula Core store

Path Aliases Reference

Alias	Resolves To
@nebula/store/*	src/nebulaKernel/redux/*
@nebula/rvw-store/*	src/runtime/rvw/redux/*
@nebula/services	src/nebulaComponents/servic es/
@nebula/authorized-rui- components	src/ramcoComponents/rui- components
@nebula/authorized-t- components	src/ramcoComponents/t- components
@kernel-styles	src/nebulaKernel/styles

Dev Server Ports Reference

Service	Port	Notes
Root-app host (nebula- runtime-platform)	:3000	Main shell

Mock API (json-server)	:3001	Auth + mock data
Remote MFEs (swift, sql-jobs, hrp, new mfe)	:3002	All remotes on same port
Preview server	:3001	Built artifact preview

FAQs

- The application is up but the components and activities listed are not same as my server.
 - This is because the root-app points to designer.ramcouat.com by default. Change the gateway URL in index.html. Additionally, validate the client ID against `REACT_APP_RIDS_AUTHORIZE_CLIENT_ID`.
- My changes are not getting reflected like in other react apps.
 - To view any change done on screen, you need to rebuild the root app and remote app.
- My screen is not navigating beyond callback URL.
 - This is probably because the callback URL displayed is not registered.